Especificações Técnicas: iScrev XBoard

Este documento define a arquitetura, as tecnologias e os padrões de projeto (Specs) adotados no desenvolvimento do **iScrev XBoard**, uma lousa digital multiplataforma desenhada para aulas, apresentações e anotações de alta performance.

---

1. Visão Geral da Arquitetura

O **iScrev XBoard** adota a abordagem *Specs Driven Development (SDD)* e foi construído para rodar de forma híbrida:

1. **PWA (Progressive Web App):** Acesso universal via navegador, rodando offline através de Service Workers (`sw.js`).
2. **Desktop App (Electron):** Encapsulamento nativo para Windows/Linux/Mac, permitindo captura de tela e microfone nativos sem prompts excessivos de segurança e isolando abas pesadas (`<webview>`).

Para manter a máxima leveza e performance, o projeto descarta frameworks inchados (como React ou Angular) em favor de **Javascript puro (Vanilla JS - ES6)**, HTML5 e CSS3 moderno (com suporte a Glassmorphism). O `<canvas>` é o núcleo do motor gráfico.

---

2. Estrutura de Diretórios (Arquitetura)

A separação de responsabilidades é rigorosa. O código que lida com o sistema operacional (Electron) é isolado da regra de negócios da Lousa (PWA Renderer).

```
/iScrev-XBoard
├── package.json            # Scripts de build (npm run dev, build, build:web)
├── main.js                 # Ponto de entrada do Electron (Main Process)
├── preload.js              # Ponte IPC segura entre Main e Renderer
├── /docs                   # Documentações e relatórios adicionais
├── /specs                  # Especificações técnicas e manuais de arquitetura
└── /src                    # PWA e Interface da Aplicação (Renderer Frontend)
    ├── index.html
    ├── manifest.json       # Configuração do PWA
    ├── sw.js               # Service Worker para PWA
    ├── /css
    │   ├── style.css       # Estilos globais e da prancheta
    │   └── ui.css          # Estilos de barra de ferramentas e modais
    ├── /js
    │   ├── /core           # Núcleo duro de performance e estado
    │   │   ├── canvas-engine.js # Motor de desenho (Bézier, eventos de pointer)
    │   │   └── history.js       # Paginação e controle de Undo/Redo
    │   ├── /modules        # Funcionalidades satélites
    │   │   ├── background-manager.js # Renderização de grades e linhas no fundo
    │   │   ├── i18n.js               # Dicionário dinâmico de tradução (PT/EN)
    │   │   ├── pdf-exporter.js       # Exportação da prancheta para PDF local
    │   │   ├── recorder.js           # MediaRecorder nativo (Tela + Microfone)
    │   │   └── ui-manager.js         # Escuta de eventos do DOM e desacoplamento
    │   └── /services
    │       └── storage.js            # Driver IndexedDB para persistência de dados
    └── /assets             # Ícones genéricos e recursos estáticos
```

---

3. Detalhamento dos Módulos Principais

A aplicação utiliza um padrão de **Classes Singleton**, acoplando instâncias únicas ao objeto global `window` após o evento de carga da página (`load`), o que simplifica injeções de dependência.

3.1. Core Engine Gráfico (`window.Engine`)

- **Tecnologia:** HTML5 `<canvas>` com `desynchronized: true` para baixa latência.
- **Mecânica:** Captura eventos unificados (`pointerdown` , `pointermove` , `pointerup`).
- **Suavização:** Ao invés de traços retos, desenha dinamicamente Curvas Quadráticas de Bézier entre os pontos coletados, calibrando a espessura da linha baseando-se na pressão física da caneta (`e.pressure`).

3.2. Gerenciamento de Estado (`window.HistoryManager`)

- O aplicativo age como um caderno. Cada folha é um "Board".
- Os estados não são salvos como pixels pesados, mas de forma **Vetorial**: coordenadas, pressões e cores, em memória (Arrays).
- Permite paginação fluida e ferramentas precisas de `undo()` (desfazer) e `redo()` (refazer) localizadas para cada página.

3.3. Banco de Dados e Persistência (`window.StorageService`)

- **Tecnologia:** API nativa IndexedDB.
- **Fluxo:** O XBoard salva a aula passivamente em background a cada 1 segundo (Debounce).
- **Recursos:** Capacidade de serializar aulas (Projetos de lousa vetorial) para a "Biblioteca de Aulas" contendo inúmeras páginas.

3.4. Motor de Gravação Nativel (`window.RecorderModule`)

- **Tecnologia:** `MediaRecorder API` , `<canvas>.captureStream(30)` e `getUserMedia` .
- **Mecânica:** Combina um canvas de fundo com o canvas de escrita num terceiro "MergeCanvas" invisível, e une ao áudio do microfone do professor. Extrai vídeos universais no formato `.webm` direto no navegador.

---

4. Estilos e Responsividade

O visual da aplicação prioriza a atenção ao centro da tela (a área de escrita). Menus são colapsáveis (Toolbar, Drawer) e modais utilizam a tag `<dialog>` do HTML5, garantindo leveza do DOM e removendo a dependência de bibliotecas obscuras de manipulação de *z-index*.

5. Próximas Atualizações (Roadmap)

- Refinamento das texturas de quadros.
- Migração completa para TypeScript (opcional) caso a escalabilidade de módulos demande tipagem forte nas estruturas do IndexedDB.